

Is chapter ka core goal hai **global scope ko sirf “bad practice” samajhne ke bajay properly understand karna**. Main isse step-by-step Hinglish mein explain karunga, especially browser, ES Modules, Node.js, Web Workers aur globalThis ke differences ke saath.

Chapter 4: Around the Global Scope

1. Global scope hota kya hai?

Scope decide karta hai ki koi variable ya function program ke **kis part se accessible** hoga.

```
1  var userName = "Ruth";  
2  
3  function greet() {  
4      console.log(userName);  
5  }
```

Yahan userName function ke bahar declare hua hai. Agar ye normal browser script hai, to ye global scope mein ho sakta hai.

Global scope ko roughly program ka **sabse outer scope** samajh sakte hain. Lekin chapter ka main point hai:

Har JavaScript environment mein file ka outermost scope real global scope nahi hota.

Browser scripts, ES Modules, Web Workers, DevTools aur Node.js sab top-level declarations ko thoda differently handle karte hain.

2. Global scope important kyun hai?

Modern JavaScript applications generally multiple files se banti hain:

```
1  app.js  
2  auth.js  
3  cart.js  
4  payment.js  
5  utils.js
```

Sawal ye hai ki in alag-alag files ka code ek doosre se communicate kaise karega?

Chapter browser applications ke liye teen main approaches explain karta hai.

Approach 1: Direct ES Modules

ES Modules mein different files explicitly import aur export ke through communicate karti hain.

```
1 // moduleOne.js
2 export function someMethod() {
3     console.log("Module one");
4 }

1 // moduleTwo.js
2 import { someMethod } from "./moduleOne.js";
3
4 someMethod();
```

Is approach mein modules ko shared global variables par depend hone ki zarurat nahi hoti.

Har module ka apna scope hota hai aur jo functionality share karni hai, use explicitly export/import kiya jata hai.

Main idea

```
1 moduleOne.js —export/import—> moduleTwo.js
```

Isme global scope cooperation ka primary medium nahi hai. Imports modules ke beech connection provide karte hain.

Approach 2: Bundler ke andar shared wrapper scope

Bundlers multiple files ko combine karke ek badi file bana sakte hain.

Kabhi combined code ko ek outer function ke andar wrap kiya jata hai:

```
1 (function wrappingOuterScope() {
2     var moduleOne = (function one() {
3         // moduleOne ka code
4     })();
5
6     var moduleTwo = (function two() {
7         function callModuleOne() {
8             moduleOne.someMethod();
9         }
10    })();
11 })();
```

Yahan:

- moduleOne real global variable nahi hai.

- moduleTwo bhi real global variable nahi hai.
- Dono wrappingOuterScope() ke local variables hain.
- Dono same outer function scope mein hone ki wajah se ek doosre ko access kar sakte hain.

Visual form mein:

```

1  Global scope
2  └─ wrappingOuterScope function
3      └─ moduleOne
4      └─ moduleTwo
5          └─ moduleOne.someMethod() access kar sakta hai

```

Ye outer function ek tarah ka **application-wide scope** provide karta hai. Ye real global scope nahi hai, lekin application ke liye global scope ke substitute ki tarah behave karta hai.

Approach 3: Global scope as shared space

Agar files:

- ES Modules nahi hain
- kisi common wrapper function mein nahi hain
- normal <script> files ke roop mein load ho rahi hain

to unke beech common shared space global scope ho sakta hai.

module1.js

```

1  var moduleOne = (function one() {
2      // moduleOne ka code
3  })();

```

module2.js

```

1  var moduleTwo = (function two() {
2      function callModuleOne() {
3          moduleOne.someMethod();
4      }
5  })();

```

Agar browser mein normal scripts ke roop mein load kiya jaye:

to dono files ke top-level var declarations global scope mein ja sakte hain.

Isliye module2.js, module1.js ke moduleOne ko access kar sakti hai.

Important detail: loading order

Conceptually, agar second script first script ke variable ko use karti hai, to first script ko pehle execute hona padega:

Agar order reverse ho aur moduleTwo immediately moduleOne ko use kare, to variable available na hone ki problem aa sakti hai.

3. Global scope koi useless cheez nahi hai

Often advice milti hai:

Global variables avoid karo.

Ye advice incomplete hai.

Better version:

Application ke har variable ko unnecessarily global mat banao.

Global scope khud useless ya inherently wrong nahi hai. JavaScript aur hosting environment already bahut saari important values global scope se expose karte hain.

JavaScript ke built-in globals

Primitive-like global values

```
1 undefined
2 Infinity
3 NaN
```

Chapter ki list mein null bhi diya gaya hai, lekin practical distinction samajhna useful hai: null ek literal value hai, normal global-object property ki tarah access ki jane wali binding nahi.

For example:

```
1 console.log(globalThis.undefined);
2 console.log(globalThis.Infinity);
3 console.log(globalThis.NaN);
```

Lekin:

```
1 globalThis.null
```

null literal ke equivalent built-in property access nahi hai.

Native constructors

```
1 Date
2 Object
3 String
4 Array
5 Number
6 Promise
```

Example:

```
1 const now = new Date();
2 const user = new Object();
```

Aapne inhe khud declare ya import nahi kiya. Ye JavaScript environment se available hain.

Global functions

```
1 parseInt()
2 eval()
```

Example:

```
1 const number = parseInt("42", 10);
2
3 console.log(number);
4 // 42
```

Global namespaces

```
1 Math
2 JSON
3 Atomics
```

Example:

```
1 Math.random();
2 JSON.stringify({ name: "Kyle" });
```

JavaScript ke related platform features

```
1 Intl
2 WebAssembly
```

Inhe bhi application global scope ke through access kar sakti hai.

4. Hosting environment ke globals

JavaScript engine ke alawa jis environment mein JS run ho rahi hai, woh bhi additional globals provide kar sakta hai.

Browser examples:

```
1 console
2 window
3 document
4 setTimeout
5 navigator
6 history
```

Example:

```
1 console.log(document.title);
2
3 setTimeout(() => {
4     console.log("Done");
5 }, 1000);
```

document, navigator aur setTimeout core JavaScript language ka inherent part nahi hain. Ye browser environment provide karta hai.

Isliye ek important distinction hai:

1 JavaScript built-ins	Hosting-environment APIs
2 -----	-----
3 Object	document
4 String	window
5 Math	navigator
6 JSON	setTimeout

JavaScript doosre environments mein bhi run ho sakti hai, lekin har environment DOM provide nahi karta.

5. Global scope as “glue”

Chapter global scope ko application ka **glue** kehta hai.

Glue ka matlab:

- different scripts ko connect karna
- built-in JavaScript functionality expose karna
- browser ya Node-specific capabilities expose karna

- application-wide values ko reachable banana

Lekin problem tab hoti hai jab global scope ko dumping ground bana diya jaye:

```
1 var userName = "Kyle";
2 var cart = [];
3 var currentPage = "home";
4 var theme = "dark";
5 var result = {};
6 var data = [];
7 var status = true;
```

Bahut saare generic global names se:

- name collisions ho sakte hain
- accidental overwriting ho sakti hai
- code ka source track karna difficult hota hai
- files ke beech hidden dependencies create hoti hain

Isliye global scope useful hai, but use carefully karna chahiye.

6. Global scope exactly kahan hota hai?

Normally hum sochte hain:

Jo code function ya block ke andar nahi hai, woh global hai.

Lekin ye har environment mein true nahi hai.

Ye code dekho:

```
1 var studentName = "Kyle";
2
3 function hello() {
4     console.log(`Hello, ${studentName}!`);
5 }
```

Iska behavior depend karega ki code:

- normal browser script hai
- ES Module hai
- Web Worker mein hai
- Node.js CommonJS module hai
- DevTools console mein type hua hai

Yahi chapter ka most important section hai.

7. Normal browser script and window

Ek normal .js file ko browser page mein load karo:

Aur app.js mein:

```
1  var studentName = "Kyle";
2
3  function hello() {
4      console.log(`Hello, ${studentName}!`);
5  }
6
7  hello();
```

Normal, non-module browser script mein:

- studentName global scope mein hota hai
- hello global scope mein hota hai
- var aur function declarations window object par properties bhi bana sakti hain

Isliye:

```
1  console.log(window.studentName);
2  // Kyle
3
4  window.hello();
5  // Hello, Kyle!
```

Yahan do related effects hue:

```
1  Global binding:      studentName
2  Global object property: window.studentName
```

Dono same value ko represent kar rahe hain.

Global environment vs global object

Ye difference bahut important hai.

Global scope/environment

Ye scope system hai jahan global identifiers exist karte hain:

```
1  studentName
2  hello
```

Global object

Ye actual object hai jo browser mein commonly window hota hai:

```
1 window.studentName
2 window.hello
```

Har global variable global object property banega, ye zaroori nahi hai.

Particularly let, const aur class ke saath difference aata hai.

8. var versus let in global scope

Normal browser script mein:

```
1 var firstValue = 10;
2 let secondValue = 20;
```

Generally:

```
1 window.firstValue;
2 // 10
3
4 window.secondValue;
5 // undefined
```

Lekin direct identifier se:

```
1 firstValue;
2 // 10
3
4 secondValue;
5 // 20
```

Matlab:

- var global binding ke saath global-object property bhi create karta hai.
- let global lexical binding create karta hai.
- let ki binding window property nahi banti.

Conceptually:

```
1 Global environment
2 |— Object-based globals
3 |   |— firstValue = 10
4 |— Lexical globals
5     |— secondValue = 20
```

Isi wajah se global scope aur global object exactly identical concepts nahi hain.

9. Globals shadowing globals

Code:

```
1 window.something = 42;
2
3 let something = "Kyle";
4
5 console.log(something);
6 // Kyle
7
8 console.log(window.something);
9 // 42
```

Isme do separate things exist karti hain:

1. window.something global object ki property hai.
2. let something global lexical binding hai.

Jab direct identifier use kiya:

```
1 something
```

to lexical binding milti hai:

```
1 "Kyle"
```

Jab explicitly window property use ki:

```
1 window.something
```

to object property milti hai:

```
1 42
```

Is situation ko simplified form mein aise dekh sakte hain:

```
1 Global lexical binding:
2 something = "Kyle"
3
4 Global object property:
5 window.something = 42
```

Lexical something, direct name lookup ke liye object property ko shadow kar raha hai.

Ye dangerous kyun hai?

Code reader naturally expect karega:

```
1 something === window.something
```

Lekin yahan:

```
1 something !== window.something
```

Isse:

- debugging confusing hoti hai
- different access styles different results dete hain
- variable ka actual source unclear ho jata hai

Chapter ki recommendation hai:

Global declaration banana hi hai to var use karo; let aur const ko block scopes ke liye reserve karo.

Ye author ki scope-organization recommendation hai. Iska purpose global lexical binding aur global-object property ke beech divergence avoid karna hai.

Modern application code mein better broader strategy usually ye hoti hai:

- unnecessary globals avoid karo
- ES Modules use karo
- values explicitly import/export karo
- global object ko directly modify karne se bacho

10. DOM elements automatically globals ban sakte hain

Browser ka ek legacy behavior hai: agar HTML element ke paas id hai, browser uske liye global reference expose kar sakta hai.

HTML:

JavaScript mein:

```
1 first;  
2 // <li id="first">Write a book</li>
```

Kyunki first valid JavaScript identifier hai, browser is element ko global name ki tarah expose kar sakta hai.

Invalid lexical identifier wala ID

Ye ID dekho:

my-todo-list direct JavaScript identifier nahi ban sakta, kyunki hyphens ko minus operators samjha jayega.

Isliye ye nahi:

```
1 my-todo-list
```

Object bracket access use kiya ja sakta hai:

```
1 window["my-todo-list"];
```

Is behavior par rely kyun nahi karna chahiye?

Because it is:

- legacy browser behavior
- implicit
- confusing
- naming conflicts create kar sakta hai
- HTML changes se JavaScript behavior affect ho sakta hai

Bad approach:

```
1 first.textContent = "Done";
```

Better explicit approach:

```
1 const firstItem = document.getElementById("first");  
2  
3 firstItem.textContent = "Done";
```

Explicit query se reader ko clearly pata hai ki DOM element kahan se aaya.

11. window.name ki odd behavior

Browser mein window.name pehle se defined property hai.

Code:

```
1 var name = 42;  
2  
3 console.log(name, typeof name);
```

Unexpected result:

```
1 "42" "string"
```

Aapne number assign kiya:

```
1 42
```

Lekin mila string:

```
1 "42"
```

Aisa kyun?

name normal blank global property nahi hai. Browser mein window.name already special predefined property hai.

Jab normal browser global scope mein likha:

```
1 var name = 42;
```

to existing window.name property ke saath interaction hota hai. window.name assigned value ko string mein convert karta hai.

Conceptually:

```
1 window.name = 42;
2
3 console.log(window.name);
4 // "42"
```

Agar let name use karein?

```
1 window.name = "Browser Value";
2
3 let name = 42;
4
5 console.log(name);
6 // 42
7
8 console.log(window.name);
9 // Browser Value
```

Yahan let name separate lexical global banayega aur direct identifier lookup ke liye window.name ko shadow karega.

Phir wahi confusing divergence:

```
1 name          -> 42
2 window.name   -> "Browser Value"
```

Lesson

Generic global names, especially built-in browser property names, conflicts create kar sakte hain.

12. Web Workers ka global scope

Web Worker JavaScript ko separate worker thread mein run karne deta hai.

Main application:

```
1 Main browser thread
2 |— UI
3 |— DOM
4 |— application code
```

Worker:

```
1 Worker thread
2 |— separate JavaScript program
```

Worker main program ke saath same global scope share nahi karta.

Worker ke paas DOM access nahi hota

Worker mein normally:

```
1 document
2 window
```

available nahi hote.

Isliye worker directly page ke DOM elements modify nahi kar sakta.

Worker aur main thread controlled messaging ke through communicate karte hain.

Worker ka global object: self

Web Worker mein global object ko generally self se reference kiya jata hai:

```
1 var studentName = "Kyle";
2 let studentID = 42;
3
4 function hello() {
5     console.log(`Hello, ${self.studentName}!`);
```

```
6  }
7
8  self.hello();
9  // Hello, Kyle!
```

Yahan:

- var studentName global-object property create karta hai.
- function hello() global-object property create karta hai.
- let studentID lexical global hai, self property nahi.

So:

```
1  self.studentName;
2  // "Kyle"
3
4  self.hello;
5  // function hello...
6
7  self.studentID;
8  // undefined
```

Lekin direct identifier available hai:

```
1  studentID;
2  // 42
```

Browser main thread aur worker ka comparison:

```
1  Normal browser script:
2  global object reference = window
3
4  Web Worker:
5  global object reference = self
```

13. DevTools Console / REPL real program jaisa kyun nahi hai?

Browser console mein aap code test karte ho:

```
1  let value = 10;
```

Aur phir dobara:

```
1  let value = 20;
```

Different consoles is code ko actual script files se differently handle kar sakte hain.

Reason ye hai ki DevTools ka primary goal hota hai:

- experimentation easy banana
- debugging convenient banana
- short snippets quickly run karna
- developer experience improve karna

Isliye console ka environment global scope ko perfectly reproduce na karke **emulate** kar sakta hai.

Kahan differences aa sakte hain?

Chapter specifically mention karta hai:

- global scope behavior
- hoisting
- top-level let and const
- kuch errors ka handling

Isliye console mein jo behavior dikha, woh actual .js file mein exactly same ho, zaroori nahi.

Important takeaway

DevTools iske liye achha hai:

- values inspect karna
- debugging
- quick experiments
- functions manually call karna

Lekin language ke nuanced scope behavior ko verify karne ke liye better hai actual file banana:

Ya module behavior check karna ho to:

Console ko final specification test environment mat samjho.

14. ES Modules ka top-level global nahi hota

Ye ES Module dekho:

```
1  var studentName = "Kyle";
2
3  function hello() {
```

```
4     console.log(`Hello, ${studentName}!`);
5   }
6
7   hello();
8
9   export { hello };
```

File ke top-level par declarations hain, lekin:

- studentName global variable nahi hai
- hello global variable nahi hai
- dono module scope mein hain

Agar module browser mein load ho:

to:

```
1 window.studentName
```

generally undefined rahega.

```
1 window.hello
```

bhi undefined rahega.

Module scope ko kaise imagine karein?

Conceptually, module ko ek hidden scope boundary ke andar samajh sakte ho:

```
1 // Conceptual model, actual transformation nahi
2 (function moduleScope() {
3     var studentName = "Kyle";
4
5     function hello() {
6         console.log(`Hello, ${studentName}!`);
7     }
8 })();
```

Actual ES Module simply function wrapper nahi hota, lekin scope understanding ke liye analogy useful hai.

Structure:

```
1 Global scope
2   └─ Module scope
3       └─ studentName
4       └─ hello
```

Module scope global scope ka child hai.

Isliye module ke andar global things access ho sakti hain:

```
1 console.log("Hello");
2 Math.random();
3 setTimeout(() => {}, 1000);
```

Lekin module ke top-level declarations automatically globals nahi banti.

Module-wide ya “module-global”

Author studentName aur hello ko informally “module-global” keh raha hai.

Meaning:

- poore module file ke andar accessible
- module ke nested functions ke liye outer binding
- lekin entire application ka global nahi
- globalThis ki property nahi

Example:

```
1 const API_URL = "/api";
2
3 function fetchUsers() {
4     console.log(API_URL);
5 }
6
7 function fetchProducts() {
8     console.log(API_URL);
9 }
```

API_URL module ke andar dono functions se accessible hai, but doosre module se automatically accessible nahi.

Share karna ho to export karna padega:

```
1 export const API_URL = "/api";
```

Aur doosri file mein:

```
1 import { API_URL } from "../config.js";
```

15. ES Modules global scope ko remove nahi karte

ES Modules global variables ki dependency reduce karte hain, lekin global scope khatam nahi karte.

Module ke andar bhi aap use karte ho:

```
1 console.log();
2 Math.random();
3 JSON.stringify();
4 setTimeout();
5 document.querySelector();
```

Ye values surrounding global environment se aa rahi hain.

Isliye:

Modules global declarations create karne ki need reduce karte hain, lekin modules khud global environment ke andar hi execute hote hain.

16. Node.js mein top level real global scope nahi hota

Node.js har .js file ko module ki tarah treat karta hai.

CommonJS example:

```
1 var studentName = "Kyle";
2
3 function hello() {
4     console.log(`Hello, ${studentName}!`);
5 }
6
7 hello();
8
9 module.exports.hello = hello;
```

Browser ki normal script mein var studentName global ho sakta tha.

Lekin Node CommonJS file mein:

- studentName module-local hai
 - hello module-local hai
 - ye global object ki properties nahi bante
-

Node conceptual wrapper

Chapter ise roughly aise imagine karta hai:

```
1 function Module(module, require, __dirname, ...) {
```

```

2    var studentName = "Kyle";
3
4    function hello() {
5        console.log(`Hello, ${studentName}!`);
6    }
7
8    hello();
9
10   module.exports.hello = hello;
11 }

```

Node internally module contents ko scoped execution context provide karta hai.

Isliye declarations wrapper ke local scope mein chali jati hain:

```

1  Node global scope
2  └─ Module wrapper scope
3     └─ studentName
4     └─ hello

```

Exact implementation details alag ho sakti hain, lekin scope samajhne ke liye ye model useful hai.

17. Node ke “globals” hamesha actual globals nahi hote

Node files mein aap use kar sakte ho:

```

1  require
2  module
3  __dirname

```

Lekin chapter ka point hai ki CommonJS context mein ye necessarily real global-scope identifiers ya global-object properties nahi hain.

Inhe Node har module ke execution context mein inject karta hai.

Wrapper analogy:

```

1  function Module(module, require, __dirname) {
2      // Aapki file ka code
3  }

```

Yahan:

- module parameter jaisa hai
- require parameter jaisa hai

- `__dirname` provided local binding jaisa hai

Isliye ye application-wide shared global variable hone ke bajay har module ko provided values hain.

18. Node mein actual global value kaise banate hain?

Node global object ko global naam se expose karta hai.

```
1 global.studentName = "Kyle";
```

Ab code:

```
1 function hello() {  
2     console.log(`Hello, ${studentName}!`);  
3 }  
4  
5 hello();  
6 // Hello, Kyle!
```

Chapter ke example mein `studentName` ko global object property ke roop mein add kiya gaya hai.

Browser comparison:

```
1 window.studentName = "Kyle";
```

Node comparison:

```
1 global.studentName = "Kyle";
```

Lekin practical application code mein global par custom state rakhna generally avoid karna better hota hai, kyunki:

- hidden dependencies create hoti hain
- tests interfere kar sakte hain
- naming collisions ho sakte hain
- code ka data source clear nahi hota

Better CommonJS sharing:

```
1 // config.js  
2 module.exports = {  
3     studentName: "Kyle"  
4 };  
  
1 // app.js  
2 const config = require("./config.js");
```

```
3
4 console.log(config.studentName);
```

Ya ESM:

```
1 // config.js
2 export const studentName = "Kyle";
1 // app.js
2 import { studentName } from "./config.js";
```

19. Same code, different environments

Code:

```
1 var value = 42;
```

Iska possible behavior:

Normal browser script

```
1 value;
2 // 42
3
4 window.value;
5 // 42
```

Browser ES Module

```
1 value;
2 // module ke andar 42
```

Lekin:

```
1 window.value;
2 // undefined
```

Web Worker classic script

```
1 value;
2 // 42
3
4 self.value;
5 // 42
```

Node CommonJS module

```
1 value;
```

```
2 // current module mein 42
3
4 global.value;
5 // undefined
```

Ye comparison chapter ka central lesson capture karta hai:

Sirf declaration dekhkar scope decide nahi kar sakte. Code kis environment aur kis loading mode mein run ho raha hai, woh bhi dekhna padega.

20. Universal global object ki problem

Different environments historically different names use karte the.

Browser:

```
1 window
```

Web Worker:

```
1 self
```

Node:

```
1 global
```

Isliye universal library code likhna difficult tha:

```
1 // Browser?
2 window
3
4 // Worker?
5 self
6
7 // Node?
8 global
```

Agar code multiple environments mein chalna hai, to kaunsa use karein?

21. Old Function() trick

Global object retrieve karne ka ek old trick:

```
1 const theGlobalScopeObject =
2     (new Function("return this"))();
```

Ye kaise kaam karta hai?

Function constructor string se function banata hai:

```
1 new Function("return this")
```

Roughly function:

```
1 function anonymous() {  
2     return this;  
3 }
```

Chapter ke according, normal invocation aur legacy non-strict behavior mein this global object ko point kar sakta hai:

```
1 const result = anonymous();
```

Aur result global object mil sakta hai.

Ye approach ugly/problematic kyun hai?

- dynamically generated code use hota hai
- samajhna difficult hai
- eval-like concerns hain
- security policies dynamic code construction ko block kar sakti hain
- modern standardized solution available hai

Isliye modern code mein is trick ki normally zarurat nahi honi chahiye.

22. globalThis: standardized solution

ES2020 se JavaScript ne standardized global object reference diya:

```
1 globalThis
```

Ye environment-independent name hai.

Browser mein:

```
1 globalThis === window  
2 // normally true in browser main context
```

Worker context mein globalThis worker global object ko reference karega.

Node mein globalThis Node ke global object ko reference karega.

Use:

```
1 globalThis.appVersion = "1.0.0";
```

Retrieve:

```
1 console.log(globalThis.appVersion);
```

Universal code:

```
1 console.log(globalThis.Math === Math);  
2 // true
```

Main benefit

Ab environment detection karke ye choose karne ki need kam ho gayi:

```
1 window  
2 self  
3 global
```

Modern code mein:

```
1 globalThis
```

use kiya ja sakta hai.

23. Pre-globalThis fallback

Chapter backward-compatible fallback dikhata hai:

```
1 const theGlobalScopeObject =  
2   typeof globalThis !== "undefined" ? globalThis :  
3   typeof global !== "undefined" ? global :  
4   typeof window !== "undefined" ? window :  
5   typeof self !== "undefined" ? self :  
6   (new Function("return this"))();
```

Execution order:

1. globalThis available hai? Use it.
 2. Nahi to global available hai? Use it.
 3. Nahi to window available hai? Use it.
 4. Nahi to self available hai? Use it.
 5. Last fallback mein Function() trick.
-

typeof kyun use hua?

Ye unsafe ho sakta hai:

```
1 if (window) {
```

```
2    // ...
3 }
```

Agar current environment mein window declared nahi hai, direct reference ReferenceError de sakta hai.

Safer:

```
1 if (typeof window !== "undefined") {
2     // window available hai
3 }
```

typeof undeclared identifier ko check kar sakta hai without immediately throwing a normal unresolved-reference error.

24. globalThis ka matlab global this nahi samajhna

Author ko globalThis name misleading lagta hai.

Reason:

- object global scope/object access ke liye hai
- name mein this hone se lag sakta hai ye universal this binding rule hai
- lekin this ka behavior separate topic hai
- globalThis ka use global object ka stable reference lene ke liye hota hai

For example:

```
1 function showThis() {
2     console.log(this);
3 }
```

Is function ka this kaise decide hoga, woh invocation aur strict mode jaise factors par depend karega.

Lekin:

```
1 console.log(globalThis);
```

standard global-object reference provide karta hai.

Dono concepts ko mix nahi karna chahiye.

25. Global variable and global object property same nahi hain

Chapter ko deeply samajhne ke liye ye distinction yaad rakho.

Case 1: var

Normal browser script:

```
1 var age = 25;
```

Usually:

```
1 age;  
2 // 25  
3  
4 window.age;  
5 // 25
```

Global binding aur global-object property connected hain.

Case 2: let

```
1 let score = 100;
```

Then:

```
1 score;  
2 // 100  
3  
4 window.score;  
5 // undefined
```

Global lexical binding exist karti hai, property nahi.

Case 3: Direct object property

```
1 window.theme = "dark";
```

Then:

```
1 window.theme;  
2 // dark
```

Normal browser global environment mein direct identifier se bhi accessible ho sakti hai:

```
1 theme;  
2 // dark
```

Lekin is behavior ko modules, Node aur different environment boundaries ke across blindly assume nahi karna chahiye.

Case 4: ES Module top-level variable

```
1  const API_URL = "/api";
```

Ye:

```
1  globalThis.API_URL
```

property create nahi karta.

API_URL sirf module scope mein hai.

26. Browser, Worker, ESM aur Node ka mental model

Normal browser script

```
1  Global scope
2  |— var declarations
3  |— function declarations
4  |— lexical let/const declarations
5  |— window global object
```

var aur functions commonly window properties se linked hote hain. let aur const global lexical bindings ho sakte hain, but window properties nahi.

Web Worker

```
1  Worker global scope
2  |— var declarations
3  |— function declarations
4  |— lexical declarations
5  |— self global object
```

DOM aur window nahi, lekin self aur selected APIs available ho sakti hain.

ES Module

```
1  Global scope
2  |— Module scope
```

```
3   |— imports
4   |— top-level var
5   |— top-level let/const
6   |— functions/classes
```

Top-level declarations module-local hain.

Node CommonJS

```
1 Node global environment
2   |— CommonJS module scope
3     |— module
4     |— require
5     |— __dirname
6     |— top-level var
7     |— top-level functions
```

Top-level declarations global nahi hote.

27. Chapter ka practical message

Rule 1: Global scope ko completely ignore mat karo

Chahe modules use karo, aap phir bhi global APIs use karoge:

```
1 console
2 Math
3 JSON
4 setTimeout
5 document
```

Global environment har JavaScript program mein relevant hai.

Rule 2: Har top-level declaration global nahi hoti

Ye code:

```
1 const value = 10;
```

Kahan run ho raha hai?

- normal browser script?
- ES Module?

- Web Worker?
- Node CommonJS?
- DevTools console?

Answer ke bina exact global behavior decide nahi kiya ja sakta.

Rule 3: Global scope aur global object alag concepts hain

```
1 let value = 10;
```

Global lexical binding ho sakti hai, but:

```
1 globalThis.value
```

necessarily exist nahi karega.

Rule 4: App variables ko unnecessarily global mat banao

Avoid:

```
1 globalThis.cart = [];  
2 globalThis.user = {};  
3 globalThis.settings = {};
```

Prefer explicit modules:

```
1 export const cart = [];  
2 export const user = {};  
3 export const settings = {};
```

Aur jahan required ho:

```
1 import { cart } from "./state.js";
```

Rule 5: DOM ID globals par rely mat karo

Avoid:

```
1 first.textContent = "Done";
```

Prefer:

```
1 const firstItem = document.getElementById("first");  
2 firstItem.textContent = "Done";
```

Rule 6: Cross-environment global object chahiye to globalThis

Modern approach:

```
1  const globalObject = globalThis;
```

Environment-specific code mein bhi names samajhna useful hai:

```
1  Browser main context -> window
2  Web Worker           -> self
3  Node legacy/common   -> global
4  Standard JS          -> globalThis
```

28. Ek comprehensive example

Normal browser script

HTML:

JavaScript:

```
1  var appName = "Todo App";
2  let currentUser = "Kyle";
3
4  function startApp() {
5      console.log(appName);
6      console.log(currentUser);
7  }
8
9  startApp();
```

Possible observations:

```
1  window.appName;
2  // "Todo App"
3
4  window.startApp;
5  // function startApp...
6
7  window.currentUser;
8  // undefined
```

But direct binding:

```
1  currentUser;
2  // "Kyle"
```

Same code as ES Module

HTML:

JavaScript:

```
1  var appName = "Todo App";
2  let currentUser = "Kyle";
3
4  function startApp() {
5      console.log(appName);
6      console.log(currentUser);
7  }
8
9  startApp();
```

Now all three module-local hain:

```
1  globalThis.appName;
2  // undefined
3
4  globalThis.currentUser;
5  // undefined
6
7  globalThis.startApp;
8  // undefined
```

Lekin module ke andar:

```
1  appName;
2  // "Todo App"
3
4  currentUser;
5  // "Kyle"
6
7  startApp();
8  // works
```

Sirf `<script type="module">` use karne se top-level scope behavior change ho gaya.

Same idea in Node CommonJS

```
1  var appName = "Todo App";
2
```

```
3 function startApp() {
4     console.log(appName);
5 }
6
7 module.exports = {
8     startApp
9 };
```

Yahan:

```
1 global.appName;
2 // undefined
```

But current module ke andar:

```
1 appName;
2 // "Todo App"
```

Doosri file access karegi through exports:

```
1 const app = require("./app.js");
2
3 app.startApp();
```

29. Common misconceptions

Misconception 1: “File ke top par hai, to global hai”

Incorrect.

ES Modules aur Node modules mein top-level declarations module-scoped hoti hain.

Misconception 2: “Har global variable window par milega”

Incorrect.

```
1 let count = 10;
2 const limit = 20;
```

Normal browser script ke global lexical scope mein ho sakte hain, lekin:

```
1 window.count;
2 window.limit;
```

properties nahi hongi.

Misconception 3: “window JavaScript ka universal global object hai”

Incorrect.

- Browser main thread mein window
 - Worker mein self
 - Node mein historically global
 - Standard cross-environment reference globalThis
-

Misconception 4: “Console mein result aa gaya, production file mein bhi same hoga”

Necessary nahi.

Console/REPL developer convenience ke liye spec behavior ko exactly reproduce na kare.

Misconception 5: “Modules use karne ke baad global scope ki knowledge useless hai”

Incorrect.

Modules ke andar bhi aap global APIs access karte ho:

```
1 console.log();
2 Math.round();
3 JSON.parse();
4 setTimeout();
```

Plus libraries aur runtime environments ko understand karne ke liye global semantics important hain.

30. Final chapter summary

Is chapter ka poora message kuch points mein:

1. **Global scope abhi bhi relevant hai**, even in modern modular JavaScript.
2. Multiple non-module scripts ke beech global scope shared communication space ban sakta hai.
3. JavaScript built-ins aur environment-provided APIs global scope se available hote hain.
4. **Global scope aur global object exactly same nahi hain.**

5. Normal browser scripts mein top-level var aur function declaration window properties create kar sakte hain.
6. Top-level let aur const global lexical bindings bana sakte hain, lekin window properties nahi.
7. DOM element IDs browser mein accidental globals create kar sakti hain, un par rely nahi karna chahiye.
8. window.name ek special browser property hai jo assigned values ko string mein convert kar sakti hai.
9. Web Worker main page se separate global scope use karta hai aur uska common global-object reference self hai.
10. DevTools Console actual JS program environment ka perfect representation nahi hai.
11. ES Module ke top-level declarations global nahi, module-scoped hote hain.
12. Node CommonJS file ka top-level bhi real global scope nahi hota, kyunki file module scope mein execute hoti hai.
13. Node mein global global object ka environment-specific reference hai.
14. Modern cross-environment standard reference:

1 globalThis

1. Best modern strategy hai globals ko unnecessarily create na karna aur modules ke through dependencies explicitly share karna.

One-line mental model

Global scope har JavaScript program ka outer foundation hai, lekin aapki file ka top-level real global scope hai ya module scope, ye runtime environment aur file-loading method decide karta hai.

Explanation approach

Maine chapter ko environment-wise break kiya, global scope aur global object ka central difference clarify kiya, aur har concept ko browser, Worker, ESM aur Node examples se connect kiya. Isse aap sirf definitions yaad nahi karenge, balki same declaration alag environments mein kyun differently behave karti hai, woh bhi samajh payenge.